



自定义List集合

北京理工大学计算机学院
金旭亮

编写自己的支持迭代访问的List集合

数据实体

 MyData	
 MyData(int, String)	
 id	int
 info	String

如果需要，可以自己写一个支持foreach迭代的集合，它需要实现Iterable<T>接口。

```
//支持迭代的集合，必须实现Iterable<T>接口
public class MyDataCollection implements Iterable<MyData>{

    //内部封装真正的数据集合
    private final List<MyData> dataList=new ArrayList<>();

    @Override
    public Iterator<MyData> iterator() {
        //偷懒，直接使用JDK为ArrayList准备好的iterator
        return dataList.iterator();
    }

    public void add(MyData data){
        dataList.add(data);
    }
}
```

使用支持foreach的自定义集合类

```
public class TestMyDataCollection {  
    public static void main(String[] args) {  
        var myCollection = new MyDataCollection();  
        //向自定义集合中添加新数据  
        for (int i = 0; i < 3; i++) {  
            myCollection.add(new MyData(i, info: "info " + i));  
        }  
        //迭代遍历集合元素  
        for (var data : myCollection) {  
            System.out.println(data);  
        }  
    }  
}
```

从零开始自定义List集合

如果需要，可以写一个类，实现List接口，从而实现你所需要的任意功能。

在本讲示例中，定义了一个MySimpleArrayList<E>，克隆了JDK中ArrayList<E>的主要功能。



MySimpleArrayList<E>关键代码

```
public class MySimpleArrayList<E> implements List<E> {  
    //使用数组封装数据  
    private E[] data = (E[])(new Object[10]);  
    //当前已保存的数据元素个数  
    private int count = 0;  
    //包容多少个元素?  
    @Override  
    public int size() {  
        return count;  
    }  
    //是否为空?  
    @Override  
    public boolean isEmpty() {  
        return count != 0;  
    }  
}
```

真正的数据，其实是放在数组中的，设为私有，外部不能直接访问。

向自定义集合尾部添加元素

```
//判断是否可以保证有这个最小的容量
private void ensureCapacity(int minCapacity) {
    if (data.length < minCapacity) {
        //扩充数组
        data = Arrays.copyOf(data, minCapacity);
    }
}

@Override
public boolean add(E e) {
    //确保有足够的容量，如果不够，就扩充底层的数组
    ensureCapacity(minCapacity: count + 1);
    data[count++] = e;
    return true;
}
```

JDK中的ArrayList是一个“动态”数组，因此，当加入的数据超过原有的数组容量时，需要扩充其容量，这个是通过“将老数组元素复制到一个更大的新数组”实现的。

在我们自己写的“ArrayList”中，也“复制”了这一作法。

在指定位置添加元素

```
//在指定的位置添加元素
@Override
public void add(int index, E element) {
    if (index > count || index < 0) {
        throw new IndexOutOfBoundsException(index + "不是一个有效的位置索引");
    }
    //在尾部追加
    if (index == count) {
        add(element);
        return;
    }
    ensureCapacity( minCapacity: count + 1);
    //将插入点后面的元素后移一个位置
    System.arraycopy(data, index, data, destPos: index + 1, length: count - index);
    //将新元素放置到空出的位置上
    data[index] = element;
    count++;
}
```

在中间插入元素时，后面的元素需要后移一个位置，并且伴随着大批元素的复制，因此，成本不菲。

提取元素

```
//按索引提取元素
@Override
public E get(int index) {
    //检查索引的有效性
    if (index < count && index >= 0) {
        return data[index];
    } else {
        throw new IndexOutOfBoundsException(index + "不是一个有效的位置索引");
    }
}
```


修改元素

```
//修改特定位置的元素
@Override
public E set(int index, E element) {
    if (index < count && index >= 0) {
        E rv = data[index];
        data[index] = element;
        return rv;
    } else {
        throw new IndexOutOfBoundsException(index + "不是一个有效的位置索引");
    }
}

@Override
public void clear() {
    //将所有数组元素全部置为null
    Arrays.fill(data, fromIndex: 0, count, val: null);
    count = 0;
}
```

由于集合中只能保存引用类型的数据，所以，清空就是简单地将所有元素全部设置为null。

删除元素

```
@Override
public E remove(int index) {
    E rv = get(index);
    //被删除元素后面的所有元素, 全部向前移动一个位置
    System.arraycopy(data, srcPos: index + 1, data, index, length: count - index - 1);
    //最后一个位置设置为null
    data[count--] = null;
    return rv;
}
```

自定义集合迭代器

//自定义迭代器

```
private class MyArrayListIterator implements Iterator<E> {  
    private int progress = 0;  
    @Override  
    public boolean hasNext() {  
        return progress < count;  
    }  
    @Override  
    public E next() {  
        return data[progress++];  
    }  
}  
  
@Override  
public Iterator<E> iterator() {  
    return new MyArrayListIterator();  
}
```

迭代器的数据来自于数组，自己只要保存一个当前访问元素的位置索引（progress字段值）就好了。

使用自定义的ArrayList

```
public class TryMySimpleArrayList {  
    public static void main(String[] args) {  
        List<String> ls = new MySimpleArrayList<>();  
        ls.add("张三");  
        ls.add("李四");  
        ls.add("王五");  
        System.out.println(ls);  
        //使用迭代器遍历  
        Iterator<String> iter = ls.iterator();  
        while (iter.hasNext()) {  
            System.out.println("> " + iter.next());  
        }  
        //使用foreach遍历  
        for (String s : ls) {  
            System.out.println("-- " + s);  
        }  
    }  
}
```



```
Run: TryMySimpleArrayList x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
MySimpleArrayList [张三, 李四, 王五]  
> 张三  
> 李四  
> 王五  
-- 张三  
-- 李四  
-- 王五  
  
Process finished with exit code 0
```

MySimpleArrayList的不足之处

```
@Override
public boolean contains(Object o) {
    throw new UnsupportedOperationException("Not supported yet.");
}

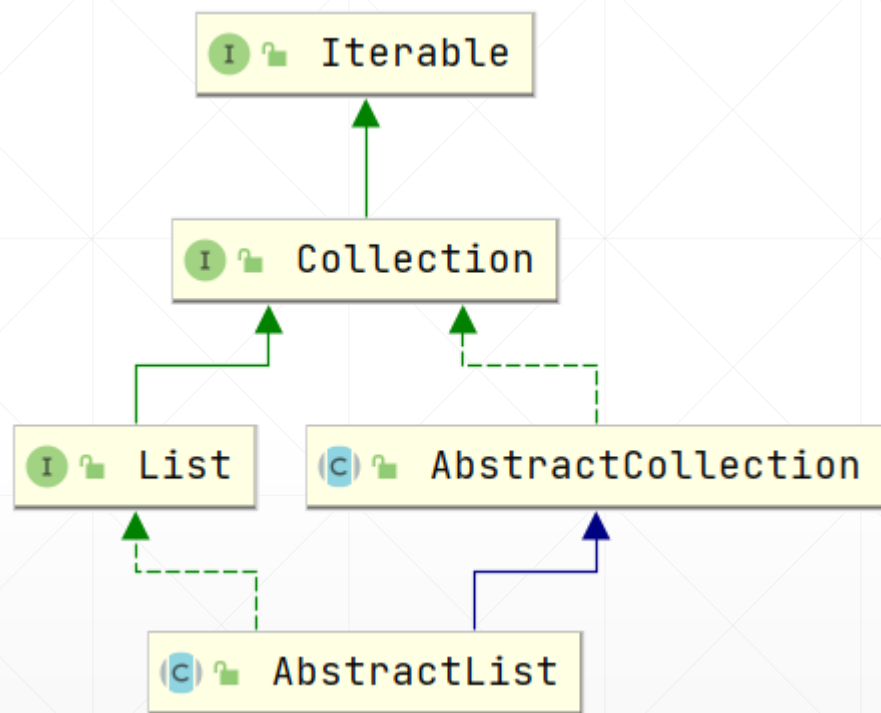
@Override
public Object[] toArray() {
    throw new UnsupportedOperationException("Not supported yet.");
}

@Override
public <T> T[] toArray(T[] a) {
    throw new UnsupportedOperationException("Not supported yet.");
}
```

JDK中的List<E>接口实在包含有太多的方法，我们自定义的集合类中没有完全实现这些方法。

如果你不想自己实现所有这些方法，有一个取巧的方法，那就是让你的集合类从AbstractList<E>派生，而不是直接实现List<E>接口。

AbstractList已经帮助我们干了许多活.....



AbstractList

m	add(E)	boolean
m	get(int)	E
m	set(int, E)	E
m	add(int, E)	void
m	remove(int)	E
m	indexOf(Object)	int
m	lastIndexOf(Object)	int
m	clear()	void
m	addAll(int, Collection<? extends E>)	boolean
m	iterator()	Iterator<E>
m	listIterator()	ListIterator<E>
m	listIterator(int)	ListIterator<E>
m	subList(int, int)	List<E>
m	equals(Object)	boolean
m	hashCode()	int

自定义集合类的最简形式

```
public class CustomArrayList<E> extends AbstractList<E> {  
    @Override  
    public E get(int index) {  
        return null;  
    }  
  
    @Override  
    public int size() {  
        return 0;  
    }  
}
```

AbstractList实现了许多最基础的List接口方法，比如，它提供了迭代器，这样，你就不需要自己写一个了。

你只需要定义好自己的内部数据存储方式，然后实现get()和size()方法，就能实现一个“固定容量”的List集合。

如果集合容量可变，重写基类的add和remove方法即可。

动手动脑



采用从`AbstractList<E>`派生的方式，克隆JDK中`ArrayList<E>`的主要功能。

提示：
请仔细阅读JDK中`AbstractList<E>`类的文档，就知道该怎样做了。